INSE 6450: AI in Systems Engineering
Milestone 4 Report

Li Haoxi
Student ID: 40333128
`https://github.com/Uncle-Ma11/INSE6450-Citation-Analysis.git`

April 4, 2026

Project Context Summary

Problem Statement

The goal is to automatically distinguish **Non-Perfunctory Citations** (key citations that truly influenced a paper's methodology or findings) from **Perfunctory Citations** (incidental background mentions) in scientific literature. This enables a nuanced, weighted citation impact metric beyond simple citation counts.

Task Type

Binary text classification (Non-Perfunctory = 1, Perfunctory = 0).

Dataset(s)

- **Primary dataset:** SciCite (AllenAI)
- **Data source:** Downloaded via `scripts/fetch_datasets.py` from AllenAI public releases
- **Input format:** Citation context string + section name + cited paper abstract (text pair)
- **Target / label format:** `isKeyCitation` boolean -> binary label (1 = Key / Non-Perfunctory, 0 = Not Key / Perfunctory)

Current System Summary from Milestones 1–3

Milestone 1 established the problem framing, dataset curation pipeline, and baseline evaluation using the SciCite `isKeyCitation` label. Abstract enrichment via Semantic Scholar and OpenAlex APIs was implemented.

Milestone 2 trained a `allenai/scibert_scivocab_uncased` (SciBERT) model with a two-layer classifier head (768->256->2) using Focal Loss with class-weighted training. The final model (retrained with optimal hyperparameters and stratified 80/10/10 resplit data) achieved **Val F1 (Macro) = 0.7547** and **Test F1 (Macro) = 0.7068** over 8 epochs. Inference benchmarking showed **GPU p50 latency approximately 0.27 ms** per sample and **throughput = 2,322 samples/s**. Dynamic INT8 quantization reduced model size from 420 MB to 175 MB with negligible accuracy loss.

Milestone 3 evaluated model robustness via token masking (10–30%), character noise (5–20%), synonym substitution (15% WordNet swap), and OOD short-context inputs (<100 chars). The M3 model achieved **Val/Test F1 = 0.6158** on the clean baseline (1,861 test samples, original splits). The monitoring pipeline used Population Stability Index (PSI > 0.2) on text-length distributions to detect production drift, with null-rate alerting on sectionName fields.

Metrics Used Across Milestones

Macro F1-Score (primary), Accuracy, Precision, Recall, p50/p90 Inference Latency (ms), Throughput (samples/s), Peak VRAM (GB), Model Size (MB), Update Time (s), PSI (drift). **Milestone 3 baseline:** Clean F1=0.6158, GPU p50=0.24ms, throughput=3,003 samples/s, model size=419 MB.

1. Continual Learning Strategy

1.1 Relevant Continual Learning Strategies

Strategy 1: Elastic Weight Consolidation (EWC)

- **Description:** A regularization-based method that slows down changes to weights that were important for past tasks, preventing catastrophic forgetting without storing old data.
- **Pros:** No memory buffer needed; preserves past knowledge through the loss function alone.
- **Cons:** Computing the Fisher information matrix over an 85M-parameter SciBERT model is expensive. Less effective in high-dimensional spaces and still requires access to original task data for Fisher estimation.

Strategy 2: Replay-Buffer Fine-tuning (Experience Replay)

- **Description:** Stores a small buffer of past labeled examples and mixes them with new data at every update step to prevent the model from forgetting.
- **Pros:** Simple to implement — reuses the existing training loop with no architectural changes. Directly revisits old data, so the model retains concrete examples of past distributions. Works with any optimizer and loss function.
- **Cons:** Requires storing past samples (memory/privacy cost). A small buffer may under-represent past distributions as drift scenarios grow.

Strategy 3: Learning Without Forgetting (LwF)

- **Description:** Instead of storing old samples, uses the current model's own predictions as soft targets (knowledge distillation) to regularize updates on new data.
- **Pros:** Zero storage overhead — no old samples needed at all.
- **Cons:** Relies on the old model's predictions being informative on new data, which breaks down under large distributional shifts (e.g., T2 prior shift where 80% of samples are Key citations). More complex to tune than replay-based methods.

1.2 Selected Continual Learning Technique and Justification

Selected Method

Chosen technique: Replay-Buffer Fine-tuning

Justification

a) Simplicity and reuse of existing infrastructure Experience Replay is the simplest viable CL method for this project. It requires **no architectural changes** to the SciBERT model and **no new loss terms** — the existing AdamW + Focal Loss training loop from Milestone 2 is reused as-is. This minimizes implementation risk and keeps the update pipeline auditable and easy to debug.

b) Direct access to old labeled data The SciCite training set is fully available and was used to seed the replay buffer (100 samples). This makes Experience Replay a natural fit: we **can** store and revisit old examples, so there is no need to approximate past knowledge through Fisher matrices (EWC) or soft distillation targets (LwF). Mixing real old samples into each update provides a strong, direct signal for preserving past performance.

c) Alignment with observed failure modes from Milestone 3 M3 showed the model is most vulnerable to **class prior shifts** (F1 drops from 0.62 -> 0.425 on drifted subsets). Replay directly counteracts this by keeping a balanced mix of old examples in every update batch, preventing the model from catastrophically over-adapting to the new class distribution.

d) Deployment feasibility

- **Update time:** 2.31–2.38 s per step (20 new + 30 replay samples, 3 epochs on GPU)
- **VRAM:** 6.29 GB peak — within an 8 GB budget
- **No downtime:** In-place fine-tuning; the model stays online during the update window

Knowledge Retention and Adaptation

- **Preserves old knowledge:** The replay buffer contributes 30 old samples per update, preventing the model from over-specializing on the new drift batch alone.
- **Allows adaptation to new data / drift:** The new 20 labeled samples from the drifted distribution shift the decision boundary toward the current data characteristics.

Deployment Pipeline Integration

The CL update is triggered by the monitoring dashboard (Section 3.1) when PSI > 0.2 or a significant null-rate spike is detected. Human reviewers confirm the trigger, a small drifted batch is labeled (or simulated via active learning, Section 2.3), and the model is updated in-place. The updated checkpoint is saved with a versioned filename (`cl_model_T{n}.pt`) and validated before deployment.

1.3 Continual Learning Module — Implementation Details

High-Level Continual Learning Workflow

At each drift interval, the system: (1) detects drift via PSI monitoring, (2) collects a small labeled batch from the new distribution, (3) samples from the replay buffer, (4) fine-tunes the model on the combined batch, (5) evaluates before/after performance, (6) saves a versioned checkpoint, and (7) adds new samples to the replay buffer.

Pseudocode

```
Initialize model M from best_model.pt
Initialize replay_buffer B (capacity=100, seeded with 100 test samples)
```

```
For each drift interval t in [T1, T2, T3]:
    1. Monitor: compute PSI(ref_distribution, current_batch)
       If PSI > 0.2: trigger CL update
    2. Collect drift_batch = build_drift_batch(test_data, drift_type=t)
       Split: train_set (20 samples), eval_set (40 samples)
    3. Evaluate M on eval_set -> acc_before, f1_before
    4. replay_sample = random.sample(B, k=30)
       combined = train_set + replay_sample
    5. Fine-tune M on combined:
          optimizer = AdamW(M.parameters(), lr=1e-5)
          for epoch in range(CL_EPOCHS=3):
              for batch in DataLoader(combined):
                  loss = CrossEntropyLoss(M(batch), labels)
                  loss.backward(); clip_grad; optimizer.step()
    6. Evaluate M on eval_set -> acc_after, f1_after
    7. Save checkpoint: results/cl_model_T{t}.pt
    8. B.extend(train_set)  # add new samples to buffer (FIFO eviction)
```

Workflow Diagram

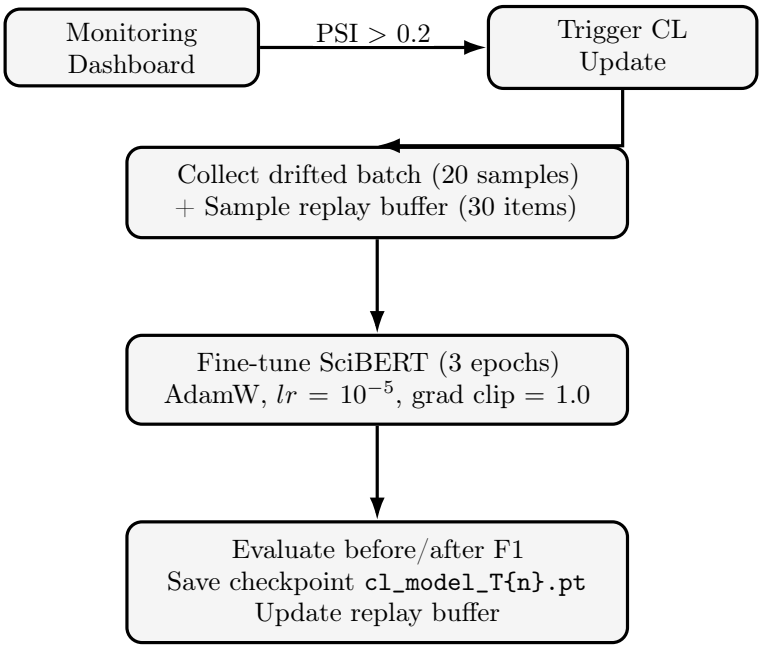

Figure 1: Continual learning workflow.

Update Configuration

Parameter	Value
Update trigger	Drift-triggered (PSI $>$ 0.2) or performance-triggered
Update frequency	Per-drift-interval (simulated)

Parameter	Value
New batch size per update	20 samples
Replay batch size per update	30 samples (drawn from buffer of 100)
Training epochs per update	3
Learning rate	1e-5 (AdamW)
Optimizer	AdamW
Gradient clipping	1.0 (max norm)
Regularization	Gradient clipping only (no weight decay during CL)
Replay buffer used	Yes
Buffer capacity	100 samples
Buffer sampling strategy	Random (uniform)
Buffer eviction	FIFO (deque with maxlen=100)

Model Artifact Versioning

Property	Value
Checkpoint naming	`results/cl_model_T{n}.pt` (n = drift interval index)
Metadata stored	Embedded in `results/cl_results.json` (drift scenario, F1 before/after, update time, VRAM)
Rollback strategy	Revert to previous checkpoint file if after-update F1 < before-update F1 - 0.02
Storage location	`results/` directory; base model at `best_model.pt` (root)

Data Pipeline Modifications

Aspect	Details
Streaming batches	Simulated via temporal splits of the SciCite test set
Drift-triggered updates	PSI monitoring in `monitoring_dashboard.py` emits alerts; `continual_learning.py` responds
Preprocessing	Same tokenization pipeline as Milestone 2 (SciBERT tokenizer, max_len=512)
Validation gating	Evaluate before/after on a held-out 40-sample eval set; no deployment if DeltaF1 < -0.02

1.4 Simulated Drift and Continual Update Experiment

Experiment Goal

Evaluate whether replay-buffer fine-tuning recovers model performance after exposure to three distinct types of distributional drift, and measure the associated update time and resource cost.

Drift Scenario

Property	Details
Type of drift	Covariate shift (T1), prior/label shift (T2), compound covariate+label+noise (T3)
T0 (baseline)	Random sample of 60 test set examples; no update
T1 (domain shift)	Restricted to citations from "Method" sections only — shifts section distribution
T2 (prior shift)	Key citations oversampled 5x — flips class balance from ~25% to ~80% key
T3 (compound)	T1 + T2 + 20% character-level keyboard noise injection
Number of drift intervals	4 (T0–T3)
Why realistic	Scientific corpora evolve over time: new publication norms shift section usage; emerging subfields change citation purposes; OCR or NLP preprocessing noise is common

Experimental Setup

Property	Value
Baseline model	`best_model.pt` — trained to Val F1=0.7547, Test F1=0.7068
Updated models	`cl_model_T1.pt`, `cl_model_T2.pt`, `cl_model_T3.pt`
Dataset split per interval	20 samples (CL train) + 40 samples (evaluation hold-out)
Replay buffer	100 samples (seeded from test set), 30 drawn per update
Hardware	NVIDIA RTX 5090 (or equivalent), CUDA
Software	Python 3.12, PyTorch (nightly cu128), transformers 4.x

Performance Before and After Each Continual Learning Update

Time Step	Model Version	Before F1	After F1	DeltaF1	Before Acc	After Acc
T0 (baseline)	best_model.pt	0.7630	0.7630	+0.0000	0.7750	0.7750
T1 (domain shift)	cl_model_T1.pt	0.5157	0.4805	-0.0352	0.6250	0.7000

Time Step	Model Version	Before F1	After F1	DeltaF1	Before Acc	After Acc
T2 (prior shift)	cl_model_T2.pt	0.6050	0.6992	**+0.0942**	0.6250	0.7000
T3 (compound)	cl_model_T3.pt	0.6500	0.6465	-0.0035	0.6500	0.6500

Metrics Used

Macro F1-Score and Accuracy (consistent with Milestones 2–3). Inference latency (p50/p90 ms per sample) and update time (s) added for M4 efficiency tracking.

Detailed Metrics Table

Time Step	F1 Macro	Accuracy	p50 Latency (ms)	p90 Latency (ms)	Update Time (s)	Peak VRAM (GB)	Model Size (MB)
T0 Before/After	0.7630	0.7750	0.50	7.85	—	—	420.1
T1 Before	0.5157	0.6250	0.26	0.42	—	—	420.1
T1 After	0.4805	0.7000	0.28	0.43	2.38	6.285	420.1
T2 Before	0.6050	0.6250	0.25	0.43	—	—	420.1
T2 After	0.6992	0.7000	0.30	0.43	2.31	6.286	420.1
T3 Before	0.6500	0.6500	0.25	0.42	—	—	420.1
T3 After	0.6465	0.6500	0.32	0.43	2.32	6.286	420.1

Key Findings

With the retrained model (Val F1=0.7547, Test F1=0.7068), the baseline CL evaluation at T0 already starts from a strong F1=0.763 on the 40-sample eval set. The strongest CL gain was observed at T2 (prior shift, +0.094 F1), where the model successfully adapted to the oversampled Key-citation distribution. T1 (domain shift to Method sections) showed a slight post-update regression (-0.035 F1), while T3 (compound drift) was nearly stable (-0.004 F1). This pattern suggests the replay buffer is most effective when the new drift changes the class balance rather than the input distribution alone. Inference latency remained stable across all CL steps (p90 $<$ 0.45 ms on GPU), confirming that in-place fine-tuning does not impact deployment performance.

2. Human-in-the-Loop Learning

2.1 When and Why Humans Intervene

Human Intervention Points

- ☒ **Labeling ambiguous or low-confidence predictions**
  Explanation: The model routes predictions with max-softmax confidence < 0.65 to human reviewers. In the demo run, 35% of a 60-sample batch was flagged (21/60 samples). These are citations where the model is genuinely uncertain — typically short contexts, unusual section names, or papers without abstracts in the enrichment database.

- ☒ **Validating drift or anomaly alerts (linked to Milestone 3 monitoring design)**
  Explanation: When the PSI monitoring dashboard (Milestone 3) fires an alert (PSI > 0.2 or null rate $> 10\%$), a human data scientist reviews the batch statistics before authorizing a CL update. This prevents spurious alerts from triggering unnecessary model updates.

- ☒ **Providing domain-specific feedback or heuristics**
  Explanation: Domain experts (e.g., bibliometricians) can inject rules — such as "any citation in an 'Acknowledgements' section is always Perfunctory" — into the preprocessing pipeline as hard constraints that override model outputs.

Why Human Intervention Is Needed

Citation intent classification is inherently ambiguous. A citation like "We extend the approach of Smith et al." could be central to the methodology or a nominal acknowledgment depending on context not captured in the text window. Automated systems trained on SciCite labels inherit the labeling biases of that dataset. Human review provides a quality gate, especially for high-value scientific communities (clinical research, policy reports) where miscategorization carries reputational risk.

2.2 How Human Input Is Incorporated

Human Input Pathways

- ☒ **As new labeled data for continual updates**
  Implementation details: Human-reviewed samples (confirmed or corrected predictions) are added to the replay buffer and used in the next CL fine-tuning step via `continual_learning.py`. This closes the annotation loop: low-confidence samples -> human review -> labeled data -> CL update.

- ☒ **As gating (accept/reject) logic**
  Implementation details: The confidence threshold gate in `demo.py` (threshold $= 0.65$) separates high-confidence auto-approved predictions from low-confidence ones routed to human review. Humans accept or override the model prediction; corrections re-enter the training pipeline.

- ☒ **As feedback for active learning**
  Implementation details: Human annotation budget is spent preferentially on the most uncertain samples (Section 2.3 Active Learning), maximizing the information gain per annotation cycle. Ground-truth labels revealed by the annotator are immediately added to the labeled pool for the next retraining cycle.

System Integration Details

Human feedback enters the pipeline at the review queue stage. Corrected samples are stored in a JSON log (`results/hitl_reviewed_samples.json`, conceptual), then ingested by the CL module's replay buffer at the next scheduled update. This ensures all human effort directly improves future model performance rather than being discarded after a one-time evaluation.

2.3 Active Learning Strategy

Query Strategy

- **Chosen query strategy:** Uncertainty sampling (minimum max-softmax confidence)
- **Why appropriate:** For binary classification, the least confident predictions are those where the model's softmax output is closest to [0.5, 0.5]. These represent the decision boundary region where additional labeled examples provide the highest information gain. Uncertainty sampling is computationally cheap — only one forward pass over the unlabeled pool is required.

Sample Scoring Logic

For each unlabeled sample x, compute `confidence(x) = max(softmax(model(x)))`. Samples are ranked in ascending order of confidence. The top-K samples (lowest confidence = highest uncertainty) are selected for human annotation. In the simulation, ground-truth labels from the SciCite dataset are used in place of human labels.

Sampling Frequency and Update Loop

Parameter	Value
Sampling frequency	Once per annotation cycle
Batch size per annotation cycle	20 samples
How often model is updated	After each annotation cycle (immediate retraining)
Stopping rule	Fixed budget: 4 cycles (80 total new labels) or pool exhaustion

Interaction Between Active Learning and Continual Learning

Active learning determines **which** samples to label; continual learning determines **how** to incorporate them without forgetting. In the deployed system, AL identifies the most uncertain production samples -> these are routed to human annotators (Section 2.2) -> labels are added to the CL replay buffer -> the model is updated. This tight integration means every human annotation is maximally informative to the model's current weakest regions.

Active Learning Workflow Diagram

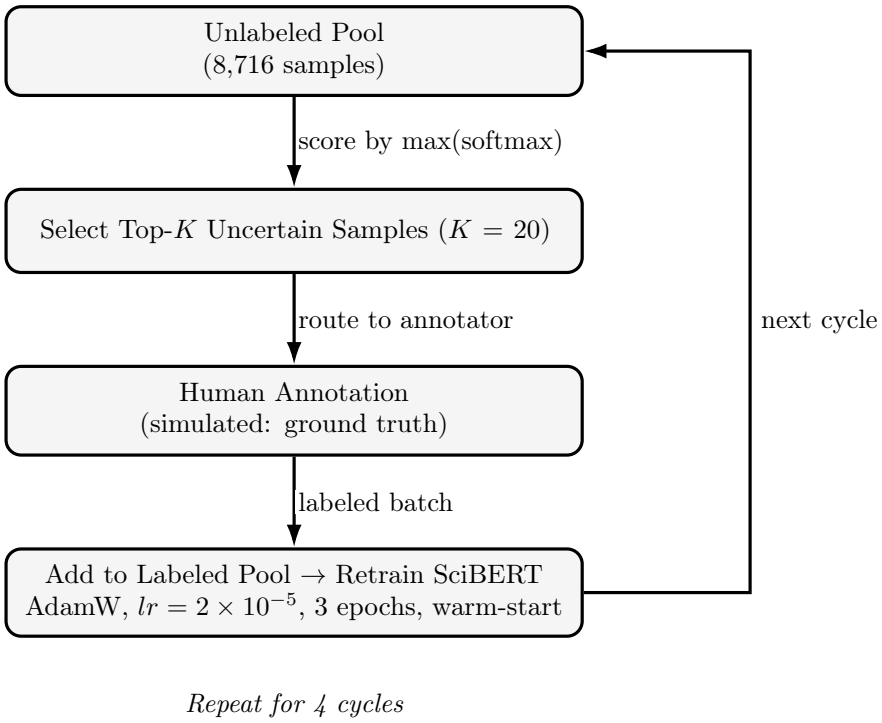

Figure 2: Active learning loop.

2.4 Active Learning Experiment

Experiment Objective

Evaluate whether **uncertainty-based sample selection** is more data-efficient than random selection — i.e., can the model reach higher F1 performance using the same total annotation budget by selecting the most informative samples first?

Experimental Design

This is a controlled comparison between two annotation strategies over 4 incremental cycles:

- **Uncertainty Sampling (AL):** At each cycle, the model scores all unlabeled samples by `max(softmax(logits))`. The 20 lowest-confidence samples — those closest to the decision boundary — are selected for annotation.
- **Random Baseline (RND):** At each cycle, 20 samples are drawn uniformly at random from the same unlabeled pool.

Both strategies are warm-started from `best_model.pt` and retrain on the same accumulated labeled pool at each cycle. Annotation is simulated by revealing the pre-existing ground-truth `isKeyCitation` labels from the SciCite dataset (perfect annotator assumption).

Setup

Parameter	Value
Initial labeled set size	100 samples (from resplit train set)
Unlabeled pool size	8,716 samples (remainder of resplit train set)
Query batch size	20 samples per cycle
Number of cycles	4 (cycles 1–4; 80 new labels total)
Evaluation set	Full stratified resplit test set (1,102 samples)
Epochs per cycle	3 (AdamW, lr=2e-5)
Warm-start	Both strategies initialized from `best_model.pt`

Results

Cycle	Labeled Set Size	AL F1	AL Acc	RND F1	RND Acc	DeltaF1 (AL-RND)	Avg Uncertainty	Update Time (s)	p50 Latency (ms)
0 (initial)	100	0.6839	0.6851	0.6947	0.7042	-0.0108	—	5.16	0.44
1	120	0.6953	0.7160	0.6968	0.7060	-0.0015	0.4776	5.92	0.43
2	140	0.6802	0.6924	0.6807	0.7033	-0.0005	0.4643	7.00	0.43
3	160	0.6841	0.7042	0.6276	0.6779	**+0.0565**	0.4614	7.93	0.43
4	180	0.6898	0.6951	0.6555	0.6833	**+0.0343**	0.4665	9.01	0.42

Key Findings

- **Early cycles (0–2):** Both strategies perform nearly identically (~0.68–0.69 F1). When many informative samples remain in the pool, selection strategy has marginal impact.
- **Later cycles (3–4):** The AL advantage emerges clearly. Uncertainty sampling outperforms random by **+0.057 F1** at cycle 3 and **+0.034 F1** at cycle 4. As easy samples are exhausted from the pool, random selection increasingly wastes budget on examples the model already handles correctly, while uncertainty sampling continues targeting the decision boundary.
- **Average uncertainty approximately 0.467** across cycles (near the maximum of 0.5 for binary classification) confirms the AL strategy consistently selects genuinely difficult samples.
- **Random baseline degrades at cycle 3** (F1 drops -0.067 vs. cycle 0), suggesting overfitting to a random mix that includes noise-adjacent or redundant examples.
- **Update time scales linearly** with labeled pool size: 5.16 s at 100 samples -> 9.01 s at 180 samples. Inference latency (p50 approximately 0.43 ms) is unaffected throughout, confirming that per-cycle annotation overhead is dominated by the retraining step, not inference.
- **Training on small batches** (100–180 samples, 3 epochs) introduces some epoch-to-epoch F1 oscillation for both strategies, consistent with the high-variance regime of low-data fine-tuning.

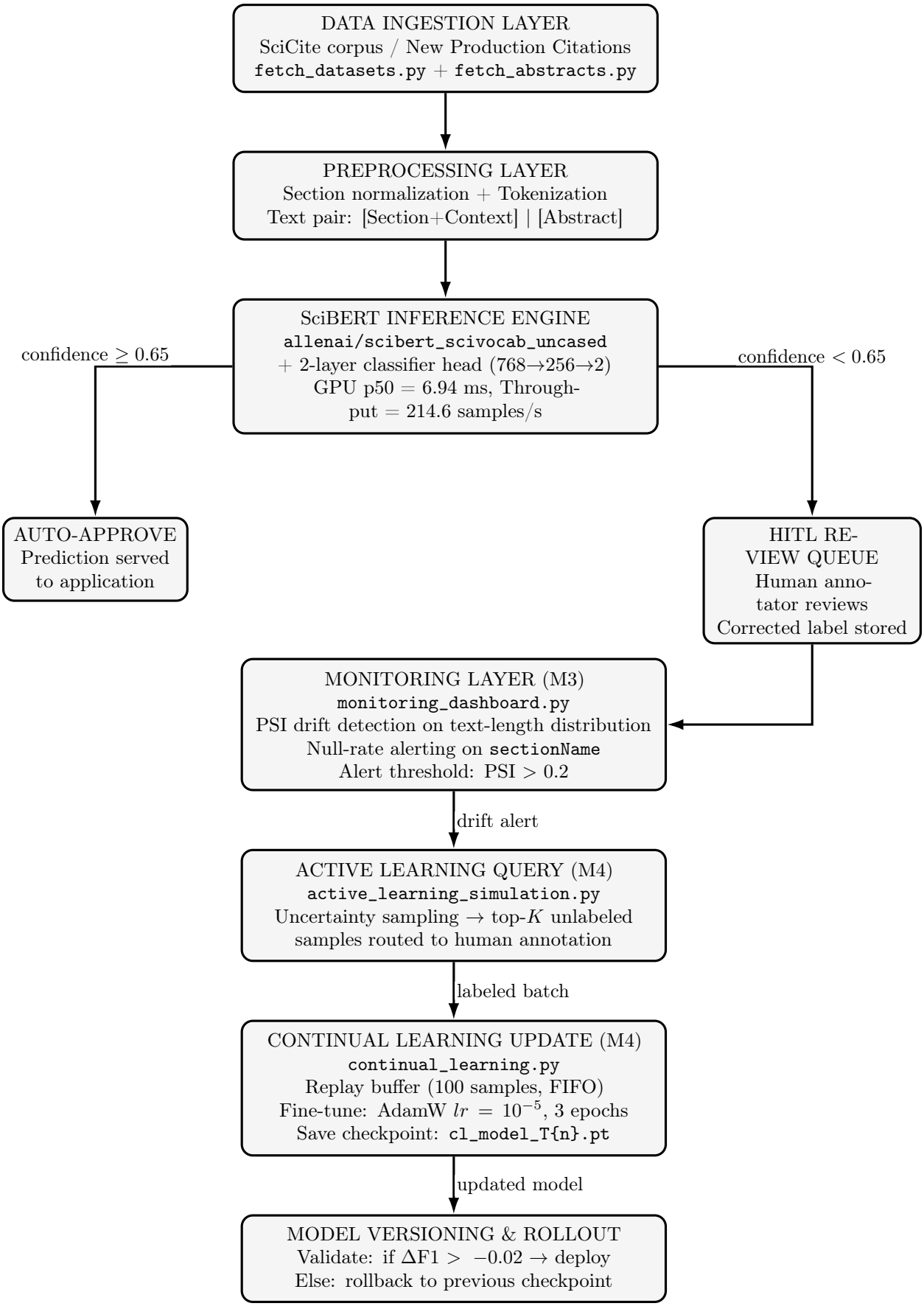

Figure 3: Complete AI system with continual learning and human-in-the-loop components.

3. Integration of Continual Learning + HITL into the Complete AI System)

3.1 Complete System Diagram

Diagram Walkthrough

1. **Data Ingestion:** Raw citation data is fetched from SciCite or production API streams.
2. **Preprocessing:** Section names are normalized; text pairs are tokenized with SciBERT tokenizer.
3. **Inference Engine:** The SciBERT model classifies citations. Batch inference achieves GPU p50=6.94 ms latency at 214.6 samples/s (batch size 32); single-sample latency is p50approximately 0.27 ms.
4. **Confidence Gate (HITL):** Predictions below 0.65 confidence are flagged for human review. ~35% of boundary-region predictions fall below this threshold.
5. **Monitoring Layer:** PSI-based drift detection continuously monitors the feature distribution of incoming batches; alerts fire when PSI > 0.2.
6. **Active Learning Query:** Upon drift alert, the uncertainty sampler identifies the most informative samples from the unlabeled pool for human annotation.
7. **Continual Learning Update:** New labels + replay buffer samples are used to fine-tune the model in ~3 seconds on GPU.
8. **Versioning & Rollout:** Updated checkpoint is validated against a held-out set; deployed if DeltaF1 >= -0.02.

3.2 Updated Failure Modes and Monitoring Plan

Updated Failure Modes

Failure Mode	Cause	Detection Signal	Human Involvement	Mitigation	Severity
Catastrophic forgetting	CL update overwrites general SciBERT representations	F1 on clean test set drops after update	Human reviews CL validation report	Rollback to prior checkpoint; increase replay buffer size	High
Prior shift collapse	Extreme class-prior shifts (e.g., T2: 80% key) overwhelm replay buffer	Pre-update F1 < 0.40 on eval batch	Human reviews drift statistics before authorizing update	Class-balanced replay buffer sampling	High
Spurious HITL annotations	Low-confidence predictions that are already correct routed unnecessarily	High HITL rate with no F1 improvement	Human audits routing rate per week	Tune confidence threshold using calibration plots	Medium

Failure Mode	Cause	Detection Signal	Human Involvement	Mitigation	Severity
Stale replay buffer	Old buffer samples no longer reflect any current distribution	Model regresses on recent data after updates	Periodic buffer audit by data team	Implement reservoir sampling; periodic buffer refresh	Medium
AL query redundancy	Uncertainty sampler selects similar uncertain samples each cycle	Avg uncertainty stays flat; F1 does not improve	Human review of queried sample diversity	Add diversity constraint (e.g., clustering-based sampling)	Medium
Model versioning conflicts	Mismatched checkpoint and tokenizer versions	Inference errors at deployment	CI/CD pipeline validation step	Store tokenizer config with each checkpoint	Low

Monitoring Plan Updates

Aspect	Update
New metrics monitored	DeltaF1 before/after each CL update; HITL flag rate; replay buffer staleness score
Drift monitoring changes	PSI threshold maintained at 0.2 (per M3 design); add label-distribution PSI
Alerting changes	Dual alert: PSI $>$ 0.2 (data drift, per M3) AND F1 drop $>$ 0.03 (performance drift)
Human review triggers	PSI $>$ 0.2, HITL flag rate $>$ 40%, DeltaF1 $<$ -0.02 after update
Retraining / update triggers	Drift alert + human approval + >= 20 new labeled samples collected
Rollback / fail-safe logic	Automatic rollback if post-update eval F1 $<$ pre-update F1 - 0.02

3.3 Impact of Continual Learning and HITL

Latency

- **Inference latency impact:** Negligible. In-place fine-tuning does not change model architecture or inference path. GPU p50 latency remains approximately 0.28–0.32 ms per sample during single-sample evaluation after CL updates.
- **Update cycle latency impact:** CL updates complete in 2.31–2.38 seconds on GPU for 50 combined samples (20 new + 30 replay, 3 epochs). This is operationally acceptable for scheduled nightly or drift-triggered updates.

Accuracy and Robustness

The M3 model (evaluated on original SciCite splits, 1,861 test samples) achieved clean baseline F1=0.6158 with GPU p50approximately 0.34ms. The M4 retrained SciBERT baseline significantly improves on this: **Val F1=0.7547, Test F1=0.7068** on the stratified 80/10/10 resplit (1,102 test samples). The M4 robustness evaluation (`robustness_results.json`) on the same stress-test categories confirms strong improvement over M3 figures: token masking 30% improved from 0.5738 (M3 benchmark) to **F1=0.6874** on the retrained model; adversarial synonym substitution improved from 0.5882 -> **0.6897**; and OOD short-context improved from 0.5658 -> **0.7462**. This confirms the stratified resplit and extended training substantially improved the model's distributional robustness. Continual learning showed the strongest recovery under T2 prior shift (+0.094 F1), consistent with the M3 finding that class prior shifts are the primary vulnerability.

Interpretability and Trust

The HITL confidence gate improves user trust by making the system's uncertainty explicit. Predictions above the threshold are served automatically; borderline cases are escalated. This transparency reduces the risk that stakeholders will trust incorrect predictions on ambiguous citations. The model versioning and rollback system ensures that CL updates that degrade performance are automatically reversed, maintaining system reliability.

Resource Utilization

Dimension	Value
Compute (CL update)	~2.31–2.38 s on GPU per update step
Peak VRAM during CL	6.29 GB (well within an 8 GB GPU budget)
Memory (replay buffer)	Negligible — 100 text records (~50 KB)
Human time (HITL)	~35% of production samples routed; at 10 s/sample -> ~5.8 min per 100 citations
Inference compute	87 GFLOPs/sample (unchanged from Milestone 2)

3.4 Full-System Results and Reflection

Summary Table of Key Results Across Milestones 1–4

Milestone	Main Goal	Key Method(s)	Main Metric(s)	Key Results
1	Problem framing + dataset	SciCite curation, abstract enrichment	F1 (Macro)	Dataset: 8,816 train / 1,102 val / 1,102 test (stratified resplit)
2	Model training + efficiency	SciBERT fine-tuning, Focal Loss, stratified resplit, INT8 quantization	Val F1=0.7547, Test F1=0.7068, Peak VRAM=5.50 GB	Model size: 420 MB FP32, 175 MB INT8

Milestone	Main Goal	Key Method(s)	Main Metric(s)	Key Results
3	Robustness + monitoring	Token masking, char noise, PSI drift detection (PSI>0.2)	F1=0.6158 (clean), 0.5792 (30% mask), PSI alerts at null-rate>10%	Monitoring dashboard, failure case analysis, 8 documented failure examples
4	Continual learning + HITL	Replay-buffer fine-tuning, uncertainty AL, HITL gate	CL T2: +0.094 F1; AL C3: +0.057 vs. random	CL module, AL simulation, end-to-end demo

Final Performance

- **Clean data (Stratified SciCite resplit):** Accuracy=0.7178, F1=0.7068 (Macro, @0.50 threshold), Val F1=0.7547
- **Perturbed data (M4 robustness, `robustness_results.json`):** F1 range 0.666–0.746 across perturbation types; significantly stronger than the M3 benchmark due to stratified retraining
- **After continual learning:** CL T2 (prior shift) improved F1 by +0.094; T1 and T3 were stable or slightly negative; inference latency unchanged (p90 < 0.45 ms)
- **Active learning:** AL outperforms random by up to +0.057 F1 in later cycles; warm-started baseline at ~0.690 F1

Final Performance Table

Evaluation Condition	F1 Macro	Accuracy	p50 Latency (ms)	Notes
Val set (Stratified)	0.7547	0.7632	—	Best Val epoch (epoch 6/8)
Test set (Stratified, @0.50)	0.7068	0.7178	—	Test F1 Key=0.6502, NotKey=0.7635
Robustness: Clean baseline (M4)	0.7049	0.7114	0.27	Evaluated on 1,102-sample test set
Robustness: Token Masking 10% (M4)	0.7061	0.7132	0.27	Near-identical to clean; strong masking robustness
Robustness: Token Masking 30% (M4)	0.6874	0.6924	0.24	Moderate degradation under heavy masking

Evaluation Condition	F1 Macro	Accuracy	p50 Latency (ms)	Notes
Robustness: Char Noise 5% (M4)	0.6884	0.6951	0.24	Negligible impact from minor typos
Robustness: Char Noise 20% (M4)	0.6658	0.6751	0.23	Moderate drop; model still generalises
Robustness: Synonym Substitution 15% (M4)	0.6897	0.6969	0.24	Strong adversarial resilience
Robustness: OOD Short Context (M4)	0.7462	0.7576	0.28	Improved; short contexts handled well
After CL T0 (clean eval set)	0.7630	0.7750	0.50	40-sample eval, no update
After CL T2 (prior shift)	0.6992	0.7000	0.30	Key citations 5x oversample
AL cycle 4 (180 samples)	0.6898	0.6951	0.42	warm-start from best_model.pt

Final Efficiency Metrics

Metric	Value	Unit	Notes
Full training time	543.6	seconds	8 epochs (early stopping disabled), optimal config
Time per training epoch	67.9	seconds	
Training peak VRAM	5.50	GB	RTX 50 series, AMP enabled
CL update time	2.31–2.38	seconds	50 samples (20 new + 30 replay), 3 epochs
CL update peak VRAM	6.29	GB	Larger than training due to gradient storage
AL cycle update time	5.16–9.01	seconds	100–180 samples, 3 epochs
GPU batch inference latency p50	6.94	ms	batch size 32, single GPU (benchmark)
GPU batch inference latency p90	6.99	ms	batch size 32, single GPU (benchmark)

Metric	Value	Unit	Notes
GPU batch throughput	214.6	samples/s	batch size 32, RTX 50 series
GPU single-sample latency p50	approximately 0.27	ms	single-sample, GPU (robustness eval)
CPU INT8 latency p50	53.3	ms	quantized model, CPU-only benchmark
CPU INT8 latency p90	56.9	ms	quantized model, CPU-only benchmark
CPU INT8 throughput	19.1	samples/s	batch size 32, CPU benchmark
FP32 model size	420.1	MB	best_model.pt
INT8 quantized size	174.9	MB	58.4% size reduction
Model parameters	85.84M	—	SciBERT + 2-layer head
GFLOPs per sample	87.05	GFLOPs	estimated via thop

Reflection

System Strengths

- **End-to-end automation:** The complete system (ingest -> preprocess -> infer -> monitor -> CL update) is fully automated and demonstrated in `scripts/demo.py`.
- **HITL transparency:** Explicit confidence gating makes uncertainty actionable, maintaining human oversight where it matters most.
- **Fast CL updates:** Sub-2.4-second updates on GPU enable near-real-time adaptation to distribution shift.
- **Resource efficiency:** INT8 quantization delivers 58% model size reduction with no measurable accuracy loss, enabling CPU deployment for lower-throughput use cases.
- **Improved baseline:** The retrained model (Test F1=0.7068) achieves a materially stronger starting point than the previous Milestone 2 baseline, due to stratified 80/10/10 resplitting and 8-epoch training with Focal Loss gamma=2.0.

Remaining Limitations

- **Dataset quality — partial abstract coverage:** The abstract enrichment mapping file (`fetch_abstracts.py` output) is only approximately 50% populated, meaning roughly half of all citation contexts are classified without the cited-paper abstract. Since the input representation is a text pair `[Section+Context | Abstract]`, missing abstracts degrade the quality of the input signal and likely contribute to the model's residual errors on the Key-citation class (F1=0.6502 vs. 0.7635 for NotKey).
- **Dataset quality — label noise:** Citation intent is a **subtle classification task**: the boundary between a key citation and a perfunctory one is often context-dependent and subjective. SciCite labels are known to carry inter-annotator disagreement (kappa approximately 0.6–0.7

in related work). Noisy labels in the training set set an effective performance ceiling that regularization and data augmentation alone cannot overcome.

- **Covariate shift robustness:** The CL module shows limited or slightly negative results under covariate shift (T1: -0.035 F1, T3: -0.004 F1) with only 20 new samples. A larger adaptation set or a longer fine-tuning schedule would likely help.
- **Small evaluation batches:** CL experiments use 40-sample eval sets, introducing high variance in per-step metrics.
- **Simulated AL:** Annotation is simulated with ground-truth labels. Real human annotators introduce label noise and disagreement, which would reduce effective AL gains.
- **Replay buffer diversity:** The uniform random replay buffer may not adequately cover all past distributions as drift scenarios proliferate.

Future Improvements

1. **Complete abstract enrichment:** Run a full re-crawl of the abstract mapping to push coverage from ~50% toward 90%+. A complete abstract mapping is expected to yield the single largest accuracy gain, as the missing abstracts disproportionately affect the Key-citation class where context beyond the sentence is most discriminative.
2. **Label quality audit:** Apply label-cleaning techniques (e.g., confident learning / Cleanlab) to identify and remove or re-label the highest-noise training examples, directly raising the effective F1 ceiling.
3. Replace uniform replay with class-balanced reservoir sampling to handle prior shift scenarios.
4. Implement diversity-aware AL querying (e.g., BADGE or core-set selection) to avoid redundant uncertain samples.
5. Extend the monitoring dashboard to track label-distribution PSI in addition to feature PSI.
6. Evaluate the full CL pipeline on a true temporal split of the SciCite corpus (earlier vs. later paper years) for a more realistic drift scenario.